

VORBEREITUNG ZUR VORLESUNG

Handout 05 – Docker Compose

Modul 3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge
Thema Deklarative Mehr-Container-Anwendungen
Dozent Dr. Ulrich Winkler
Semester 5. Semester · Informationstechnik

Vom Befehls-Wust zur einen Datei

In Übung 04 haben Sie Netzwerk, Datenbank, Adminer und `log_tool` mit vier langen `docker run`-Befehlen von Hand gestartet. Das ist fehleranfällig und schlecht zu wiederholen. **Docker Compose** beschreibt denselben Aufbau **deklarativ** in einer Datei `compose.yaml`: Sie sagen, *was* laufen soll, nicht *wie* man es Schritt für Schritt startet.

Imperativ vs. deklarativ

- **Imperativ** (Übung 04): eine Folge von Befehlen, die den Zustand herstellen.
- **Deklarativ** (Compose): eine Beschreibung des *gewünschten Zustands*. `docker compose up` sorgt dafür, dass genau dieser Zustand existiert.

Der deklarative Ansatz ist reproduzierbar, versionierbar (die Datei liegt im Repo) und dokumentiert die Architektur gleich mit.

Aufbau einer `compose.yaml`

```
services:          # die einzelnen Container
  db:
    image: postgres:17
    environment: { POSTGRES_DB: logbuch, ... }
    volumes: [ pgdaten:/var/lib/postgresql/data ]
  adminer:
    image: adminer
    ports: [ "8080:8080" ]
  log_tool:
    build: .        # eigenes Image aus dem Dockerfile bauen
    depends_on: { db: { condition: service_healthy } }
volumes:
  pgdaten:         # benanntes Volume
```

Das gemeinsame **Netzwerk legt Compose automatisch an**; die Dienste erreichen sich über ihren Dienstnamen (`db`, `adminer`). Genau die Service Discovery aus Handout 04 – nur ohne manuelles `docker network create`.

depends_on und Healthchecks

`depends_on` allein wartet nur, bis ein Container **gestartet** ist – nicht, bis er **bereit** ist. Ein frischer PostgreSQL braucht aber einen Moment. Deshalb bekommt `db` einen **Healthcheck** (`pg_isready`), und `log_tool` wartet mit `condition: service_healthy`, bis die Datenbank wirklich Anfragen annimmt. Das beseitigt den Verbindungsfehler beim ersten Start dauerhaft.

Healthcheck

Ein Healthcheck ist ein kleiner Test, den Docker regelmäßig im Container ausführt. Erst wenn er erfolgreich ist, gilt der Dienst als „healthy“, – und abhängige Dienste dürfen starten.

Profile: dauerhaft laufende vs. Abruf-Dienste

`db` und `adminer` sollen dauerhaft laufen. `log_tool` ist dagegen ein Kommandozeilen-Werkzeug, das man nur bei Bedarf aufruft. Damit es nicht bei `up` mitstartet, steht es in einem **Profil** (`profiles: ["werkzeug"]`) und wird mit `docker compose run --rm log_tool ...` auf Abruf gestartet.

Die wichtigsten Compose-Befehle

Befehl	Wirkung
<code>docker compose up -d</code>	Stack im Hintergrund starten (baut fehlende Images)
<code>docker compose ps</code>	Status der Dienste
<code>docker compose logs db</code>	Logs eines Dienstes
<code>docker compose run --rm log_tool add "..."</code>	Abruf-Dienst einmalig ausführen
<code>docker compose down</code>	Stack stoppen (Volumes bleiben)
<code>docker compose down -v</code>	Stack stoppen und Volumes löschen